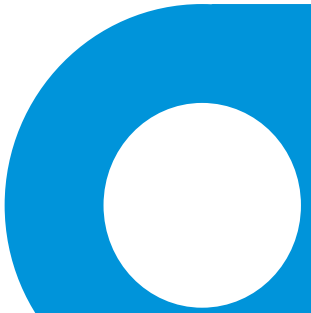


"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING



Generic Programming

Advanced Programming

FCSE 2026

Motivation: Casting exception

```
public class OldBox {
    Object data;
    public OldBox(Object data) {
        this.data = data;
    }
    public Object getData() {
        return data;
    }
}
```

An explicit cast is required to avoid a compile-time error.

```
OldBox intBox = new OldBox(42);
int x = (Integer) intBox.getData();
OldBox strBox = new OldBox("Hi");
String s = (String) strBox.getData();
```

Even with the cast the program can compile successfully but may still crash at runtime.

```
int y = (Integer) strBox.getData();
intBox = strBox;
```

ClassCastException!

```java
public class IntBox {
    Integer data;
    public IntBox(Integer data) {
        this.data = data;
    }
    public Integer getData() {
        return data;
    }
}
```

```java
public class StrBox {
    String data;
    public StrBox(String data) {
        this.data = data;
    }
    public String getData() {
        return data;
    }
}
```

```java
IntBox intBox = new IntBox(42);
int x = intBox.getData();
StrBox strBox = new
StrBox("Hi");
String s = strBox.getData();
int y = (Integer)
strBox.getData();
intBox = strBox;
```

```java
public class FooBox {
    Foo data;
    public FooBox(Foo data) {
        this.data = data;
    }
    public Foo getData() {
        return data;
    }
}
```

The errors will be caught by the compiler

Infinitely many possible classes

Passing Parameters to Methods: An Analogy

```
public abstract class Sum {
  public static int sum_0_1() {
    return (0+1);
  }
  …
  public static int sum_15_22() {
    return (15+22);
  }
  …
}
```

```
public abstract class NewSum {
  public static int
    sum(int m, int n) {
    return (m+n);
  }
}
```

Methods Receive Parameters

```
public class Main {
 public static void
 main(String[] nu) {
    int j = Sum.sum_0_1();
    …
    int k = Sum.sum_15_22();
  }
}
```

Bad – an infinite number of methods

```
public class NewMain {
  public static void
  main(String[] nu) {
    int j = NewSum.sum(0,1);
    …
    int k = NewSum.sum(15,22);
  }
}
```

Passing Parameters to Methods

Java Generics: The Core Concepts

- Parameterized Types
  - Allows classes, interfaces, and methods to operate on **types as parameters**.
  - This means you can define a component once and then use it with many different data types.
- Stronger Type Safety at Compile Time
  - The primary goal is to enforce type constraints when you write the code, not when you run it.
  - The compiler checks for type consistency and flags errors immediately.
- Elimination of Explicit Casts
  - By knowing the specific type a collection or method is working with, the compiler can insert the necessary casts for you.
  - This prevents `ClassCastException` errors at runtime, which are a common source of bugs in non-generic code.
- Enabling Generic Algorithms
  - Developers can implement algorithms that are reusable, type-safe, and work across different collections and custom types.

Generic classes

```
public class OldBox {
   Object data;
   public OldBox(Object data) {
      this.data = data;
   }
   public Object getData() {
      return data;
   }
}
```

- We want the box to contain a "specified" class – abstractly represented
- Object doesn't work well, as we've seen in previous examples
- Solution: Parameterizing the definition of a class

```
public class Box<E> {
    E data;
    public Box(E data) {
        this.data = data;
    }
    public E getData() {
        return data;
    }
}
```

- `E` refers to a specific type
- The constructor takes an object of type `E`, not just any object
- To use a class, `E` must be replaced by a specific class

How to use generic class

```
public class Box<E> {
    E data;
    public Box(E data) {
        this.data =
data;
    }
    public E getData() {
        return data;
    }
}
```

```
Box<Integer> intBox =
        new Box<Integer>(42);
int x = intBox.getData();//no cast needed

Box<String> strBox =
        new Box<String>("Hi");
String s = strBox.getData();//no cast needed
```

The following statements won't compile:

```
String s = (String) intBox.getData();
int y = (Integer) strBox.getData();
intBox = strBox;
```

Runtime errors have become compile-time errors

Using Generic Classes

- A generic class definition (a class using type parameters like <T>) is saved and compiled exactly like a standard class.
  - Behind the Scenes: Java achieves this through a process called Type Erasure. After compilation, the type parameters (<T>) are essentially removed (erased) and replaced with their bounded type (usually Object).
  - Result: At runtime, only one version of the generic class exists, saving memory and ensuring backward compatibility.
- Once the generic class is compiled, it must be instantiated by specifying the concrete type that will replace the parameter placeholder (<T>).
  - Requirement: The specific type argument must be provided before the class can be used to create objects in the program.

```
Sample<String> object =
    new Sample<String>();
```

  - Since Java 7, the compiler can usually infer the type argument from the left side of the assignment which simplifies instantiation:

```
Sample<String> object = new Sample<>();
```

Defining Parameterized Class

- A Class Definition with a Type Parameter

```
public class Sample<T>
   {
        private T data;
        public void setData(T newData)
        {
            data = newData;
        }
        public T getData()
        {
            return data;
        }
   }
```

T is a parameter for a type

Defining Parameterized Classes (Generics)

- A class defined to accept type arguments is called a Generic Class or a Parameterized Class.
  - The type specified inside the angle brackets is known as the Type Parameter or Formal Type Parameter.
- The type parameter(s) must be specified within angle brackets (<>) immediately following the class name in the class declaration.
- While any non-keyword identifier can be used, standard practice is to use uppercase letters for type parameters:
- Once defined, the type parameter (T) acts as a full-fledged placeholder for a concrete type throughout the entire class definition.
  - It can be used to **declare instance variables** (fields).
  - It can be used as the **return type of methods**.
  - It can be used as the **parameter type for method arguments**.

Generic Ordered Pair Class (1/3)

```
public class Pair<T>
{
    private T first;
    private T second;
    public Pair()
    {
        first = null;
        second = null;
    }
    public Pair(T firstItem, T secondItem)
    {
        first = firstItem;
        second = secondItem;
    }
```

Constructor headings do not include the type parameter in angle brackets

Generic Ordered Pair Class (2/3)

```
public void setFirst(T newFirst)
{
    first = newFirst;
}
public void setSecond(T newSecond)
{
    second = newSecond;
}
public T getFirst()
{
    return first;
}
public T getSecond()
{
    return second;
}
```

Generic Ordered Pair Class (3/3)

```
    public String toString()
    {
        return ( "first: " + first.toString() + "\n"
                 + "second: " + second.toString() );
    }
    public boolean equals(Object otherObject)
    {
        if (otherObject == null )
            return false;
        else if (getClass() != otherObject.getClass())
            return false ;
        else
        {
            Pair<T> otherPair = (Pair<T>)otherObject;
            return (first.equals(otherPair.first)
                    && second.equals(otherPair.second));
        }
    }
}
```

Using the Pair Class (1/3)

```
import java.util.Scanner;
public class GenericPairDemo
{
    public static void main(String[] args)
    {
        Pair<String> secretPair =
                new Pair<String>("Happy", "Day");

        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter two words:");
        String word1 = keyboard.next();
        String word2 = keyboard.next();
        Pair<String> inputPair =
                new Pair<String>(word1, word2);
```

Using the Pair Class (2/3)

```
if (inputPair.equals(secretPair))
    {
        System.out.println("You guessed the secret words");
        System.out.println("in the correct order!");
    }
    else
    {
        System.out.println("You guessed incorrectly.");
        System.out.println("You guessed");
        System.out.println(inputPair);
        System.out.println("The secret words are");
        System.out.println(secretPair);
     }
}
```

Using the Pair Class (3/3)

```
Enter two words:
two words
You guessed incorrectly.
You guessed
first: two
second: words
The secret words are
first: Happy
second: Day
```

The generic constructor has no parametric type

- Although the class name of a generic class also contains the parameter type, the parameter type is not used in the constructor definition header

```
public Pair<T>()
```

- A constructor can use the type parameter, such as T , as the type for a parameter for the constructor, but the constructor heading does not include the type parameter in angular brackets

```
public Pair(T first, T second)
```

- When you instantiate a generic class by specifying a type for the type parameter, you do specify the type in angular brackets when writing the constructor name

```
Pair<String> pair =
    new Pair<String>("Happy", "Day");
Pair<String> pair =
    new Pair<>("Happy", "Day");
```

or relying on type inference using the diamond

```
Pair pair = new Pair();
```

Raw type
(identical to Pair<Object>)

Raw type

- A raw type is the name of the generic class or interface without any type of argument.

```
public class Box<T> { public void set(T t)
{ /* ... */ } // ... }
```

- To create a parameterized type from a **Box<T>**, a specific type should be specified for the formal parametric type **T**:

```
Box<Integer> intBox = new Box<>();
```

- If the type is omitted, a raw type of **Box<T>** will be created:

```
Box rawBox = new Box();
```

- **Box** is a raw type of the generic **Box<T>** type. A non-generic class or interface type is not a raw type.

Raw type

```
Box<String> stringBox = new Box<>();
Box rawBox = stringBox; // OK

Box rawBox = new Box(); // rawBox is a raw type of
                        Box<T>
Box<Integer> intBox = rawBox; // warning: unchecked
                              conversion

Box<String> stringBox = new Box<>();
Box rawBox = stringBox; rawBox.set(8); // warning:
                       unchecked invocation to set(T)
```

Restrictions and Limitations of Generics

- Generic type arguments must be Reference Types (classes, interfaces, or arrays).
  - They cannot be primitive types like int, double, or char.
  - Since Java has automatic boxing/unboxing, this is not a major restriction.
- **Primitive types cannot be used as Type Arguments.**

Illegal Usage

```
List<int> myList;
Box<char> myBox;
```

Required Usage (Legal)

```
List<Integer> myList;
Box<Character> myBox;
```

Restrictions and Limitations of Generics

- Restriction on Instance Creation (No new T())
- You cannot instantiate a generic type parameter.

```
T object = new T();
T[] a = new T[10];
T[] a = new (T[])Object[10];
```

- You cannot create an array whose element type is a parameterized type.

```
Pair<String>[] a =
  new Pair<String>[10];
```

Using the Ordered Pair Class and Automatic Boxing (1/2)

```
import java.util.Scanner;
public class GenericPairDemo2
{
    public static void main(String[] args)
    {
        Pair<Integer> secretPair =
                new Pair<Integer>(42, 24);

        Scanner keyboard = new Scanner(System.in);
        System.out.println("Enter two numbers:");
        int n1 = keyboard.nextInt();
        int n2 = keyboard.nextInt();
        Pair<Integer> inputPair =
                new Pair<Integer>(n1, n2);
```

Automatic boxing allows you to use an **`int`** *argument for an* **`Integer`** *parameter.*

Using the Ordered Pair Class and Automatic Boxing (2/2)

```
    if (inputPair.equals(secretPair))
    {
        System.out.println("You guessed the secret numbers");
        System.out.println("in the correct order!");
    }
    else
    {
        System.out.println("You guessed incorrectly.");
        System.out.println("You guessed");
        System.out.println(inputPair);
        System.out.println("The secret numbers are");
        System.out.println(secretPair);
    }
}
}
```

```
Enter two numbers:
42 24
You guessed the secret numbers
in the correct order!
```

A Class Definition Can Have More Than One Type Parameter

- A class definition can have any number of type parameters
- The multiple type parameters are listed in angular brackets just as in the single type parameter case but are separated by commas.

Multiple Type Parameters (1/3)

```
public class TwoTypePair<T1, T2>
    {
        private T1 first;
        private T2 second;
        public TwoTypePair()
        {
            first = null ;
            second = null ;
        }
        public TwoTypePair(T1 firstItem, T2 secondItem)
        {
            first = firstItem;
            second = secondItem;
        }
        public void setFirst(T1 newFirst)
        {
            first = newFirst;
        }
```

Multiple Type Parameters (2/3)

```
 public void setSecond(T2 newSecond)
{
    second = newSecond;
}
public T1 getFirst()
{
    return first;
}
public T2 getSecond()
{
    return second;
}
public String toString()
{
    return ( "first: " + first.toString() + "\n"
            + "second: " + second.toString() );
}
```

Multiple Type Parameters (3/3)

```
    public boolean equals(Object otherObject)
    {
        if (otherObject == null )
            return false ;
        else if (getClass( ) != otherObject.getClass( ))
            return false ;
        else
        {
            TwoTypePair<T1, T2> otherPair =
                    (TwoTypePair<T1, T2>)otherObject;
            return (first.equals(otherPair.first)
                    && second.equals(otherPair.second));
        }
    }
}
```

The first equals *is the* equals *of the type* **T1**.
The second equals *is the* equals *of the type* **T2**.

Using a Generic Class with Two Type Parameters

```
import java.util.Scanner;
    public class TwoTypePairDemo
    {
        public static void main(String[] args)
        {
            TwoTypePair<String, Integer> rating =
                    new TwoTypePair<String, Integer>("The Car Guys", 8);
            Scanner keyboard = new Scanner(System.in);
            System.out.println(
                    "Our current rating for " + rating.getFirst());
            System.out.println(" is " + rating.getSecond());
            System.out.println("How would you rate them?");
            int score = keyboard.nextInt();
            rating.setSecond(score);
            System.out.println(
                    "Our new rating for " + rating.getFirst());
            System.out.println(" is " + rating.getSecond());
        }
    }
```

```
Our current rating for The Car Guys
 is 8
How would you rate them?
9
Our new rating for The Car Guys
 is 9
```

A Generic Class Cannot Be an Exception Class

You cannot create a generic class from the **`Exception`**, **`Error`**, **`Throwable`**, or any class derived from **`Throwable`**

```
public class GEx<T> extends Exception
```

will cause a compilation error

Generic Interfaces

- An interface can have one or more type parameters
- The details and notation is the same as for classes with type parameters

Generic Methods

- **Generic methods** are methods that introduce **their own type parameters**, independent of the class they reside in.
- A generic method can be a member of an ordinary or generic class and have its own parametric type. They provide flexibility and type safety even when operating within non-generic classes.
- The **type parameter** of the generic method **is local** and applies to the method itself, not to the class

Generic Methods

- A method becomes generic by declaring one or more type parameters immediately after the method's modifiers (like **`public`**, **`static`**) and before the return type.
- **`public static <T> T genMethod(T[] a)`**
- When a generic method is called, the specific data type is placed in angle brackets before the method name.
- **`String s = NonG.<String>genMethod(c);`**
- Typically, this is not required because Java can perform Type Inference, meaning it can automatically determine the concrete type based on the arguments passed.

Generic Methods

```
public class Foo {
    //Foo is not parameterized
    public <T> T aMethod(T x) {
        //will not compile without <T>
        //to indicate that this is a
        //parameterized method.
        return x;
    }
    public static void
    main(String[] args) {
        Foo foo = new Foo();
        int k = foo.aMethod(5);
        String s = foo.aMethod("abc");
    }
}
```

Fixed foo class and aMethod() variables

```
public class Bar<T> {
    //Bar is parameterized
    public T aMethod(T x) {
        return x;
    }
    public static void
    main(String[] args) {
        Bar<Integer> bar =
                new Bar<Integer>();
        int k = bar.aMethod(5);
        String s = bar.aMethod("abc");
        //Compilation error here
    }
}
```

The object Bar<T> is fixed and we are bound by a concrete T.

Generic Method in Generic Class

```
public class FooBar<T> {
        //FooBar class is parameterized with T
        //the method is also parameterized with V
        public <V> V aMethod(T x, V y) {
            return y;
        }
        public static void
        main(String[] args) {
            FooBar<Integer> bar =
                    new FooBar<Integer>();
            int k = bar.aMethod(5, 6);
            String s = bar.aMethod(7, "abc");
            System.out.println(k + s);
        }
    }
```

Generic Classes in Methods

- A parameterized class **is a type** like any other class
- It can be used to specify type of parameter as well as the return type in a method

```
Box<String> aMethod(int i, Box<Integer> b) { … }
```

- If the class is parameterized, the parameter type can be used as the data type in a declaration

```
public class Box<E> {
    E data;
    public Box(E data) {
        this.data = data;
    }
    public E getData() {
        return data;
    }
    public void copyFrom(Box<E> b) {
        this.data = b.getData();
    }
}//We have added an infinite number of types of Boxes
//by writing a single class definition
```

Inheritance and Subtyping with Generics

Understanding how inheritance works with generic classes is vital, as it introduces **non-intuitive rules** that differ from standard inheritance.

1. General Inheritance Rules

- A generic class can participate in the standard Java inheritance model:
  - A generic class can extend a non-generic class.
  - A generic class can extend another generic class (potentially restricting its type parameters).
  - Standard Rule Applies: An object of a derived class type (Subclass) is still a valid type of its superclass (Superclass).

Inheritance and Subtyping with Generics

2. The Core **Restriction: Invariance**

- This is the most critical rule that distinguishes generic subtyping from regular subtyping.
  - The Rule: **Generics are Invariant.**
  - If you have two classes, **`A`** and **`B`**, where **`B`** is a subclass of **`A`** (**`B extends A`**), this **inheritance relationship does not automatically extend to generic types** using those classes.
- Formally: For any generic class **`G`**, there is NO relationship between **`G<A>`** and **`G<B>`** (regardless of the relation between classes **`A`** and **`B`**)

Deriving Generic Classes (1/3)

```
public class UnorderedPair<T> extends Pair<T>
{
    public UnorderedPair()
    {
        setFirst( null );
        setSecond( null );
    }
    public UnorderedPair(T firstItem, T secondItem)
    {
        setFirst(firstItem);
        setSecond(secondItem);
    }
```

Deriving Generic Classes (2/3)

```
    public boolean equals(Object otherObject)
    {
        if (otherObject == null )
            return false ;
        else if (getClass() != otherObject.getClass())
            return false ;
        else
        {
            UnorderedPair<T> otherPair =
                    (UnorderedPair<T>)otherObject;
            return (getFirst().equals(otherPair.getFirst())
                && getSecond().equals(otherPair.getSecond()))
                || (getFirst().equals(otherPair.getSecond())
                && getSecond().equals(otherPair.getFirst()));
        }
    }
}
```

Deriving Generic Classes (3/3)

```
public class UnorderedPairDemo
   {
       public static void main(String[] args)
       {
           UnorderedPair<String> p1 =
                   new UnorderedPair<String>("peanuts", "beer");
           UnorderedPair<String> p2 =
                   new UnorderedPair<String>("beer", "peanuts");
           if (p1.equals(p2))
           {
               System.out.println(p1.getFirst() + " and " +
                       p1.getSecond() + " is the same as");
               System.out.println(p2.getFirst() + " and "
                       + p2.getSecond());
           }
       }
   }
```

```
peanuts and beer is the same as
beer and peanuts
```

Bounded Type Parameters (Restricting Generics)

1. The Need for Bounded Types

- In many generic scenarios, you need to call methods that are guaranteed to exist on the type parameter (`T`). Standard generics only guarantee that `T` is an `Object`, which prevents access to specific methods (e.g., methods for comparing or mathematical operations).
- Solution: Bounded Type Parameters restrict the types that can be substituted for the type variable, ensuring necessary functionality is present.

2. Defining Bounds

- The extends keyword is used to specify a bound. Note that extends is used for both classes and interfaces in this context.

```
<T extends BoundaryType>
```

Bounding to an Interface

- If you need to guarantee that type **`T`** can be compared to other objects:

```
// T must be a type that implements the Comparable interface.
public class RClass<T extends Comparable<T>> {
// Now methods like T.compareTo() can be safely called.
}
```

- Attempting to instantiate this class with a type that does not implement **`Comparable`** (e.g., new **`RClass<MyUncomparableClass>()`**) will result in a compile-time error.

Bounding to a Class

- If you need T to be a specific class or any of its subclasses:

```
// T must be Class1 or any class derived from Class1.
public class ExClass<T extends Class1> {
    // Now methods defined in Class1 can be called on
objects of type T.
}
```

Multiple Bounds

- You can restrict a single type parameter by multiple interfaces and/or a single class using the ampersand (&) symbol.
- Rules for Multiple Bounds:
  - A bound can include at most one class.
  - If a class is used, it must be listed first.
  - All subsequent bounds must be interfaces.

```
// T must be a subclass of BaseClass AND implement
Serializable AND Cloneable
public class MultiBound<T extends BaseClass &
Serializable & Cloneable> { ... }
```

Multiple Type Parameters

- If a class uses multiple distinct type parameters, each must be bounded separately:

```
// T1 extends Class1, while T2 extends Class2 AND
   implements Comparable
   public class Two<T1 extends Class1,
           T2 extends Class2 & Comparable<T2>> { ... }
```

Bounded Type Parameter

```
public class Pair<T extends Comparable>
{
    private T first;
    private T second;
. . .
public T max()
{
    if (first.compareTo(second) <= 0)
        return first;
    else
        return second;
}
```

Bounded Type Parameter

```
public class Pair<T extends Comparable< T >>
{
    private T first;
    private T second;
. . .
public T max()
{
    if (first.compareTo(second) <= 0)
        return first;
    else
        return second;
}
```

Restricting parametric types

```
public class MathBox<E extends Number> extends
Box<Number> {
    public MathBox(E data) {
        super(data);
    }
    public double sqrt() {
        return Math.sqrt(getData().doubleValue());
    }
}
```

<E extends Number> means that the parameter type of the **MathBox** must be a subclass of the **Number** class.

```
new MathBox<Integer>(5);//Legal
new MathBox<Double>(32.1);//Legal
new MathBox<String>("No good!");//Illegal
```

Restricting parametric types

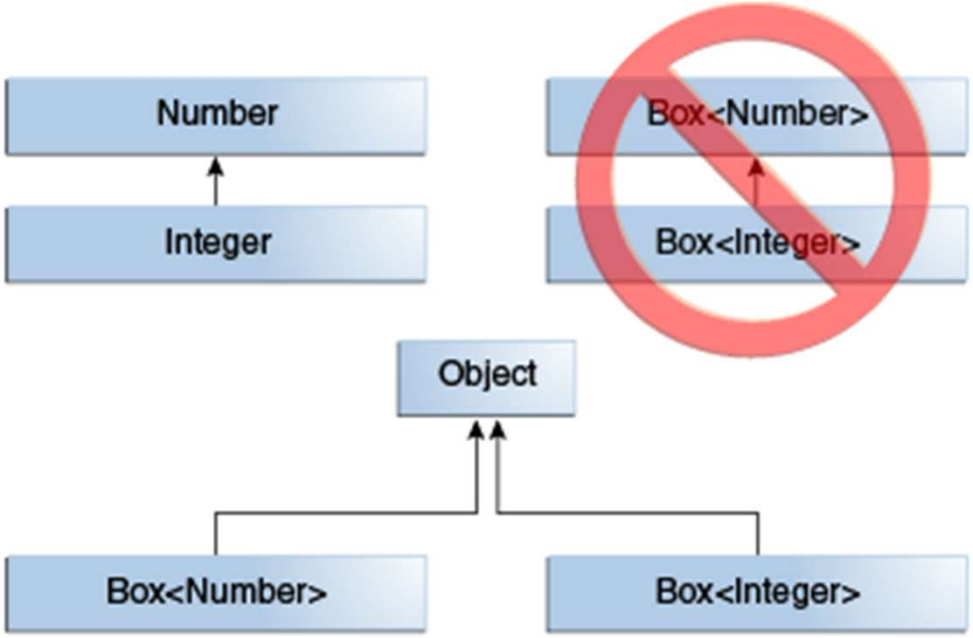
- In the definition of a parameterized class, the parametric type is a valid data type:

```
public class OuterClass<T> {
    private class InnerClass<E extends T> {
      …
    }
  …
}
```

Generics and subtyping

Although the following command seems reasonable, the compiler gives an error message:

```
Box<Number> numBox = new Box<Integer>(31);
```



Rule: Even if Class **B** is a subclass of Class **A** (B extends A), then **Generic<B> is NOT a subtype of Generic<A>.**

The Invariance Rule (Why Subtyping Fails)

- The compiler enforces invariance to maintain Type Safety and prevent a critical runtime error known as Heap Pollution.
- The error message given by the compiler is "*Incompatible Type*"
- A **numBox** can only contain an object of type **Number** and not an object of type **Integer** (although **Integer** is a class derived from **Number**)
- **Box<T>** is not a class derived from **Box<E>**, even if **T** is derived from class **E**

```
//Consider the following lines of code
Box<String> strBox = new Box<String>("Hi");//1
Box<Object> objBox = strBox;//2 – compilation error
    objBox.setData(new Object());//3
String s = strBox.getData();//4–an Object to a String!
```

Generics and subtyping

- Consider the following hypothetical scenario if the assignment were allowed:

```
// Assume a simple generic Box class:
// class Box<T> { private T data; public void setData(T d) {...} public T
getData() {...} }

// Scenario (Code Line-by-Line):

Box<String> strBox = new Box<>("Initial String"); // 1. Box holds String (OK)

// !!! THIS LINE IS ILLEGAL AND CAUSES THE COMPILATION ERROR !!!
// If this were allowed, strBox and objBox would point to the SAME object.
// Box<Object> objBox = strBox; // 2. (Hypothetically allowed assignment)

// If line 2 was allowed:
objBox.setData(new Object());   // 3. We insert a raw Object into objBox.
// This object is NOT a String.

String s = strBox.getData();    // 4. We retrieve the content via strBox
(which expects a String).
// The JVM tries to cast the stored Object (from line 3)
// into a String, resulting in a **ClassCastException at runtime!**
```

Wildcards: The Unbounded Type (<?>)

- Wildcards allow flexibility in method parameters, without sacrificing type safety.
- An **Unbounded Wildcard** (**<?>**) means "a reference to a generic type where the type parameter is unknown."
  - It can match any type argument (e.g., String, Integer, Animal).
  - It is used when the method logic does not depend on the specific type being processed but needs to work with a generic object.
- The following statements are legal:

```
// b1 can hold a Box parameterized by any type.
Box<?> b1 = new Box<Integer>(31);

// b2 can also hold a Box parameterized by any type.
Box<?> b2 = new Box<String>("Hello");

// This assignment is now legal because both b1 and b2
// are declared with the same common wildcard type:
b1 = b2;
```

Wildcards: Capabilities and Restrictions

The key purpose of **`Box<?>`** is to allow reading and manipulation of generic objects without knowing their internal type.

Operation	Allowed?	Rationale
Reading Data	**Yes**	You can always retrieve the item, but its type is only known as Object.
Writing Data	**No**	You cannot call a setter method (setData(T)) because the compiler does not know the required type T and cannot guarantee safety.
Calling Object Methods	**Yes**	Any object can safely call methods from java.lang.Object.

WildcardLimitationDemo

```
public class WildcardLimitationDemo {
    public static void main(String[] args) {
        // Create a Box specifically for Integers
        Box<Integer> integerBox = new Box<>(123);
        // Assign it to an unbounded wildcard reference.
        // The compiler now only knows it's a "Box of something."
        // It has forgotten that the "something" is an Integer.
        Box<?> wildcardBox = integerBox;
        // --- LIMITATION: WRITING DATA (NOT ALLOWED) ---
        // This is the most important limitation.
        // The compiler will stop you from putting *anything* into the box
        // because it cannot guarantee it's the correct type.
        // The following lines will ALL cause a COMPILE-TIME ERROR:
        // wildcardBox.setData(456);       // ERROR: Can't be sure it wants an Integer
        // wildcardBox.setData("Hello");    // ERROR: Can't be sure it wants a String
        // wildcardBox.setData(new Object()); // ERROR: Can't be sure it wants an
Object
        // The only exception is 'null', which is a valid value for any type.
        wildcardBox.setData(null); // This is allowed.
        System.out.println("Data after setting to null: " + wildcardBox.getData());
```

WildcardLimitationDemo

```
        // --- ALLOWANCE: READING DATA (AS OBJECT) ---

        // You can read data, but it's only guaranteed to be an Object.
        Object data = wildcardBox.getData();
        System.out.println("Data read as Object: " + data);

        // This would be a COMPILE-TIME ERROR, as the compiler
        // no longer knows the data is an Integer.
        // Integer intData = wildcardBox.getData(); // ERROR

        // --- ALLOWANCE: CALLING OBJECT METHODS ---

        // Methods from java.lang.Object are always safe to call.
        String boxString = wildcardBox.toString();
        int hashCode = wildcardBox.hashCode();

        System.out.println("toString() result: " + boxString);
        System.out.println("hashCode() result: " + hashCode);
    }
}
```

Wildcard Capture

- The compiler can recognize the type of b1 from the right-hand side of the assignment statement
  - When the compiler encounters an unbounded wildcard <?>, it treats it as a temporary, specific type that is **unknown but fixed** for the scope of the operation.
  - In the example
    ```
    Box<?> b1=new Box<Integer>(31);
    ```
    the compiler "captures" the type `Integer` from the right side, but `b1` remains usable only as a general `Box<?>`. This allows the compiler to enforce safety rules, such as preventing illegal writes, based on the captured but unknown type.

Wildcards

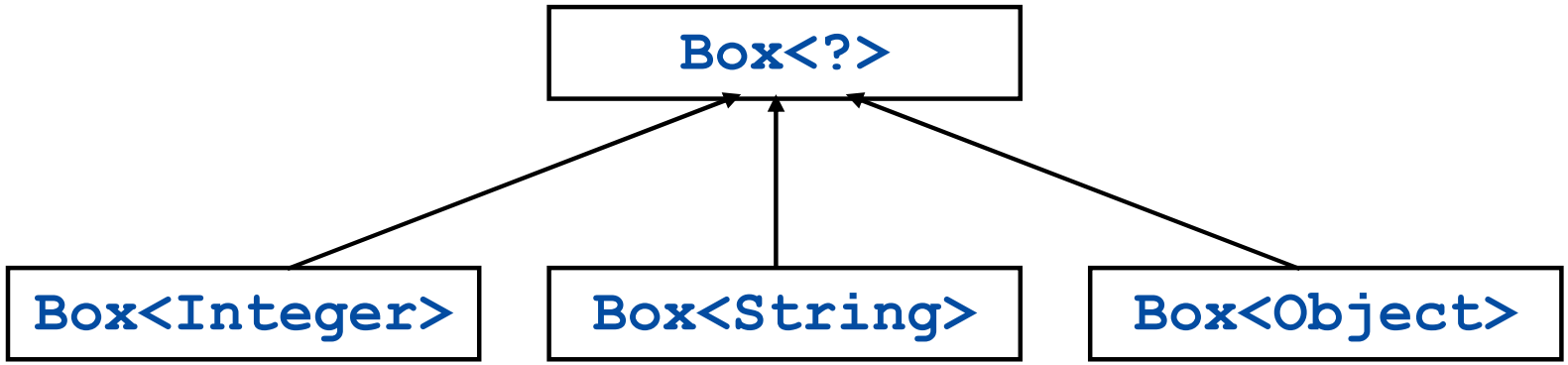
- We would like to write a method for printing any Box.

```
public static void printBox(Box<Object> b) {
    System.out.println(b.getData());
}
public static void main(String[] args) {
    Box<String> strBox = new Box<String>("Hi");
    printBox(strBox);//compilation error
}

public static <T> void printBox(Box<T> b) {
    System.out.println(b.getData());
}//parameterized method
public static void printBox(Box<?> b) {
    System.out.println(b.getData());
}//using unbounded wildcard
```

Unbounded Wildcards (<?>)

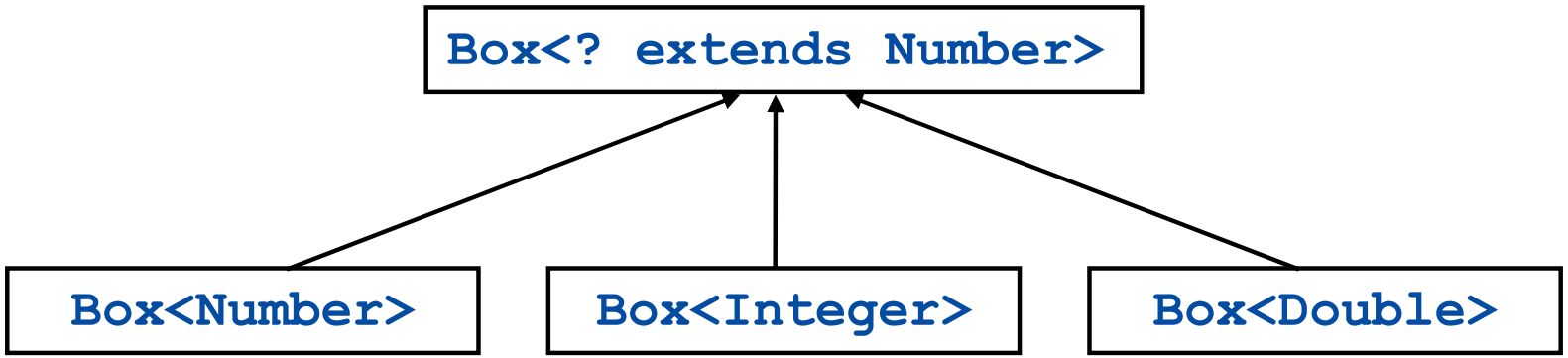
Represents a type parameter that can be any type. **`Box<?>`** is the supertype of every other fully specified generic type **`Box<T>`** for each **`T`**.



Unbounded Wildcards are ideal for writing utility methods that are completely independent of the parameter type like reading data without needing to know the specific content type (e.g., printing the contents of any list).

Upper Bounded Wildcards (<? extends T>)

Restricts the unknown type to be **`T`** or a subtype (derived class) of **`T`**.
`Box` of any type that is a subtype of **`Number`**.



`<? extends T>` is called "***upper bounded wildcard***" because it defines a type that is restricted by the (super) class E.
Useful when you only need to read (consume) items from the generic structure.

```
// numBox can hold Box<Integer>, Box<Double>, Box<Number>, etc.
Box<? extends Number> numBox = new Box<Integer>(31);
// You can read the data (it's guaranteed to be a Number), but you
cannot safely add new data.
```

Upper Bounded Wildcards

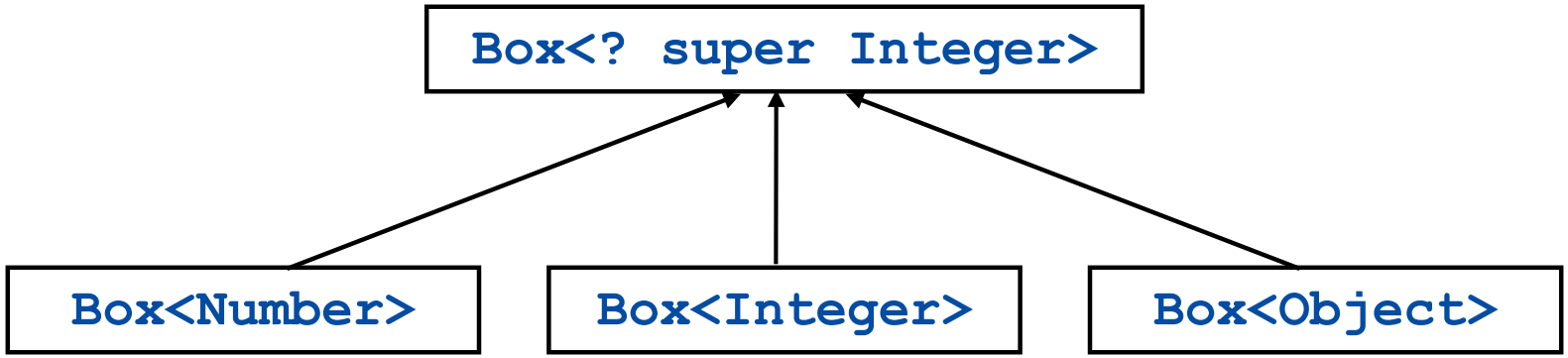
```
public class Box<E> {
        public void copyFrom(Box<E> b) {
            this.data = b.getData();
        }
    }
    //We have seen this earlier
    //We can rewrite copyFrom() so that it can take a box
    //that contains data that is a subclass of E and
    //store it to a Box<E> object

    public class Box<E> {
        public void copyFrom(Box<? extends E> b) {
            this.data = b.getData();//b.getData() is a
            //subclass of this.data
        }
    }

    Box<Integer> intBox = new Box<Integer>(31);
    Box<Number> numBox = new Box<Number>();
    numBox.copyFrom(intBox);
```

Lower Bounded Wildcards (<? super T>)

Restricts the unknown type to be **`T`** or a supertype (base class) of **`T`**.
"Box of any type that is a super-type of **`Integer`**".



`<? super E>` is called "***lower bounded wildcard***" because it defines a type that is restricted by the (sub) class E.
Useful when you only need to write (produce) items into the generic structure.

```
// intBox can hold Box<Integer>, Box<Number>, Box<Object>, etc.
Box<? super Integer> intBox = new Box<Object>();
// You can add Integers (or subtypes of Integer), but retrieving data is
unsafe (returns only Object).
```

Lower Bounded Wildcards for Consumers

- Consider a generic Box<E> class where we want to copy the contents (E) of the current box (this) into another box (b).
- Let's assume the copyTo() method copies data in the opposite direction from the copyFrom() method. copyTo() copies the data from the host object into the object passed as an argument.

```
public void copyTo(Box<E> b) {
    b.data = this.getData();
}
```

- The previous code is legal as long as the host object and object b are boxes of the same type. But b could be a box of objects instantiated from a super-class of E.
- To allow the destination box (b) to accept the data, we must define its type parameter to be a supertype of the data being copied (E). This makes the destination box a Consumer.

```
public void copyTo(Box<? super E> b) {
    b.data = this.getData();
    //b.data() is a superclass of this.data()
}
Box<Integer> intBox = new Box<Integer>(31);  // Source: Holds an Integer (E = Integer).
Box<Number> numBox = new Box<Number>();  // Destination: Holds a Number (Supertype of E).
intBox.copyTo(numBox);  // Legal: The copyTo method allows
                                    // numBox (as Box<? super Integer>) to consume the Integer data.
```

Mnemonic for Wildcards: PECS

- A common mnemonic to remember when to use which bounded wildcard:
  - **P**roducer **E**xtends: If the container will ***produce*** (give you) elements, use extends. (**`<? extends T>`**)
  - **C**onsumer **S**uper: If the container will ***consume*** (take/store) elements, use super. (**`<? super T>`**)


Questions?